The Hierarchical Subgraph-DFS Algorithm

Formal Design, Complexity Analysis, and
Cross-Domain Application Study

Stephan Epp

March 13, 2026

Contents

1 Introduction

Modern computational problems across science, industry, and engineering share a fundamental structural property: the systems of interest are not flat, but *hierarchically composed* – they are systems of systems, networks of networks, or graphs of graphs. The human connectome is a graph of neural sub-circuits; a gene-regulatory network is a graph of pathway modules; a logistics network is a graph of regional sub-networks; an industrial automation system is a graph of interacting Petri-net models.

The central question that unifies these domains is:

> *Given a collection $\mathcal{G} = \{G_1, G_2, \ldots, G_N\}$ of graphs, which members stand in a subgraph relationship to which other members, and how does structural reachability propagate through that relationship?*

This paper introduces the **Hierarchical Subgraph-DFS (HS-DFS) algorithm**, a two-phase procedure that (i) constructs a *meta-graph* $\mathcal{M}$ whose nodes are the individual graphs and whose directed edges encode detected subgraph containment, and (ii) executes a Depth-First Search on $\mathcal{M}$ to generate a *DFS forest* that exposes the full hierarchical structure of the collection. The pairwise subgraph detection in Phase (i) is performed by the **Subgraph Algorithm** [1]: a novel signature-based procedure that represents graphs as adjacency matrices, encodes each column as a unique integer via a polynomial hashing scheme, and decides containment by testing n cyclic rotations of the column-signature sequence using Longest Common Subsequence (LCS) comparison – achieving $\mathcal{O}(n^3)$ time without any exponential permutation search.

The choice of DFS in Phase 2 is deliberate and algorithmically justified: unlike Breadth-First Search, DFS yields a *DFS forest* – a spanning forest that classifies every edge as a tree, back, forward, or cross edge, thereby revealing cycles, hierarchical depth, and strong-connectivity structure that BFS cannot expose [2].

Contributions. The main contributions of this paper are:

(i) A complete formal specification of the **Subgraph Algorithm** [1] – a signature-based, cyclic-rotation oracle for pairwise subgraph detection running in $\mathcal{O}(n^3)$, serving as the concrete oracle component of HS-DFS.

(ii) A rigorous formal definition of the HS-DFS algorithm with full pseudocode, integrating the Subgraph Algorithm as its oracle.

(iii) A comprehensive worst-case and average-case complexity analysis establishing $\mathcal{O}(n^5)$ time complexity for dense meta-graphs.

(iv) A systematic survey of application domains illustrating how HS-DFS provides a unifying algorithmic primitive across neuroscience, genomics, logistics, industrial automation, software engineering, and beyond.

2 Preliminaries and Formal Definitions

Definition 2.1 (Graph)**.** A *graph* $G = (V, E)$ consists of a finite set of vertices V and a set of edges $E \subseteq V \times V$. We write $|G|$ for $|V|$.

Definition 2.2 (Subgraph)**.** A graph $H = (V_H, E_H)$ is a *subgraph* of $G = (V_G, E_G)$,

written $H \subseteq G$, if and only if $V_H \subseteq V_G$ and $E_H \subseteq E_G$. We additionally call H an *induced subgraph* if $E_H = \{(u, v) \in E_G \mid u, v \in V_H\}$.

Definition 2.3 (Subgraph-Isomorphism)**.** A graph $H = (V_H, E_H)$ is *subgraph-isomorphic* to $G = (V_G, E_G)$, written $H \hookrightarrow G$, if there exists an injective mapping $\varphi : V_H \to V_G$ such that $(u, v) \in E_H \Rightarrow (\varphi(u), \varphi(v)) \in E_G$.

Definition 2.4 (Subgraph Oracle)**.** A *subgraph oracle* $\mathcal{S}$ is an algorithm that, given two graphs H and G each of size at most n, decides whether $H \hookrightarrow G$. In this paper, $\mathcal{S}$ is instantiated with the **Subgraph Algorithm** (Section 3), which achieves $T_{\mathcal{S}}(n) = \mathcal{O}(n^3)$ via column signatures and cyclic rotations of adjacency matrices.

Remark 2.5. Subgraph isomorphism is NP-complete in general [3]. The $\mathcal{O}(n^3)$ bound of the Subgraph Algorithm is attained by restricting vertex re-labellings to n cyclic rotations instead of $n!$ arbitrary permutations. This preserves sequential edge structure and is sufficient for the structural-similarity applications targeted by HS-DFS. For fully general subgraph isomorphism (arbitrary label permutations), exponential running times remain necessary in the worst case.

Definition 2.6 (Graph Collection and Meta-Graph)**.** Let $\mathcal{G} = \{G_1, \ldots, G_N\}$ be a collection of N graphs. The *meta-graph* $\mathcal{M} = (\mathcal{G}, \mathcal{E})$ is the directed graph with node set $\mathcal{G}$ and edge set

$$\mathcal{E} \;=\; \big\{(G_i, G_j) \;\mid\; G_i \hookrightarrow G_j,\; i \neq j\big\}.$$

Definition 2.7 (DFS Forest)**.** Given a directed graph $\mathcal{M}$, a *DFS forest* $\mathcal{F}$ is the spanning forest produced by Depth-First Search on $\mathcal{M}$. Each edge in $\mathcal{M}$ is classified as a *tree edge*, *back edge*, *forward edge*, or *cross edge* with respect to $\mathcal{F}$.

3 The Subgraph Algorithm

The HS-DFS framework requires a reliable, efficient pairwise subgraph oracle $\mathcal{S}$. This section presents the **Subgraph Algorithm** [1] in full: a concrete procedure that decides $G \hookrightarrow G'$ by exploiting *column signatures* of adjacency matrices and *cyclic vertex re-labelling*, achieving $\mathcal{O}(n^3)$ time and $\mathcal{O}(n^2)$ space without recourse to exponential permutation enumeration.

3.1 Adjacency Matrix Representation

Definition 3.1 (Adjacency Matrix)**.** The *adjacency matrix* of a graph $G = (V, E)$ with $n = |V|$ vertices is the matrix $A \in \{0, 1\}^{n \times n}$ defined by

$$A_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Definition 3.2 (Subgraph-Decision Problem)**.** Given graphs G and G' with adjacency matrices $A \in \{0, 1\}^{n_A \times n_A}$ and $B \in \{0, 1\}^{n_B \times n_B}$, the *subgraph-decision problem* asks whether $G \hookrightarrow G'$ and determines the *containment relation*:

- $B \supseteq A$: discard G; retain G' as the more informative graph.
- $A \supseteq B$: discard G'; retain G.

- $A = B$ (isomorphic): retain either.
- Neither: retain both graphs as structurally incomparable.

3.2 Column Signatures

Definition 3.3 (Column Signature)**.** Let $M \in \{0,1\}^{n\times n}$ be an adjacency matrix. The *column signature* of column $j \in \{0,\dots,n-1\}$ is

$$\sigma_j(M) \;=\; \sum_{i=0}^{n-1} M_{ij}\cdot 2^i \;+\; j\cdot 2^n.$$

The *signature sequence* of M is $\boldsymbol{\sigma}(M) = [\sigma_0(M),\ \sigma_1(M),\ \dots,\ \sigma_{n-1}(M)]$. The *row-component sequence* strips the column-index weight: $r_j(M) = \sigma_j(M) \bmod 2^n$, i.e., $r_j(M) = \sum_{i=0}^{n-1} M_{ij}\cdot 2^i$.

Lemma 3.4 (Injectivity of Column Signatures)**.** *The column signature function $\sigma_j : \{0,1\}^{n\times n} \to \mathbb{N}$ is injective: distinct column vectors at distinct positions always receive distinct signatures.*

Proof. The row component $\sum_{i=0}^{n-1} M_{ij}\cdot 2^i$ encodes the binary column vector uniquely as an integer in $[0, 2^n-1]$. The column-index weight $j\cdot 2^n$ maps each $j \in \{0,\dots,n-1\}$ to the interval $[j\cdot 2^n,\ (j+1)\cdot 2^n - 1]$, which is disjoint from all other such intervals. Hence two (column-vector, position) pairs produce the same signature only if both the binary vector and the column index coincide, establishing injectivity. □

Example 3.5. Let $n = 4$. For column $j = 0$ with column vector $[1,0,1,0]^\top$:

$$\sigma_0 = 1{\cdot}2^0 + 0{\cdot}2^1 + 1{\cdot}2^2 + 0{\cdot}2^3 + 0{\cdot}2^4 = 5, \qquad r_0 = 5.$$

For column $j = 1$ with the identical vector $[1,0,1,0]^\top$:

$$\sigma_1 = 1{\cdot}2^0 + 0{\cdot}2^1 + 1{\cdot}2^2 + 0{\cdot}2^3 + 1{\cdot}2^4 = 21, \qquad r_1 = 5.$$

The signatures differ ($5 \neq 21$) despite identical column vectors, confirming injectivity. The equal row components ($r_0 = r_1 = 5$) reflect that both columns encode the same edge pattern – used later to detect structural matches across different vertex positions.

3.3 Cyclic Rotation Instead of Full Permutation

A cornerstone of the Subgraph Algorithm is the replacement of all $n!$ vertex permutations by only n cyclic rotations, reducing complexity from super-exponential to cubic while preserving structural correctness.

Definition 3.6 (Cyclic Rotation)**.** For a sequence $\mathbf{s} = [s_0, s_1, \dots, s_{n-1}]$ and offset $k \in \{0,\dots,n-1\}$, the *k-th cyclic rotation* is

$$\mathrm{rotate}_k(\mathbf{s}) = [s_k,\ s_{k+1},\ \dots,\ s_{n-1},\ s_0,\ \dots,\ s_{k-1}].$$

Example 3.7. For $n = 4$ columns with row-component sequence $[r_0, r_1, r_2, r_3]$:

$$\begin{aligned} k = 0: &\quad [r_0,\ r_1,\ r_2,\ r_3] \\ k = 1: &\quad [r_1,\ r_2,\ r_3,\ r_0] \\ k = 2: &\quad [r_2,\ r_3,\ r_0,\ r_1] \\ k = 3: &\quad [r_3,\ r_0,\ r_1,\ r_2] \end{aligned}$$

This yields only 4 cases to examine rather than $4! = 24$ permutations.

3.4 Subgraph Criterion via LCS

Theorem 3.8 (Subgraph Criterion)**.** *Let G and G' have row-component sequences $\mathbf{r}^A = [r_0^A, \ldots, r_{n_A-1}^A]$ and $\mathbf{r}^B = [r_0^B, \ldots, r_{n_B-1}^B]$. Then $G \hookrightarrow G'$ if and only if there exists $k \in \{0, \ldots, n_B - 1\}$ such that*

$$\mathrm{LCS}\big(\mathbf{r}^A,\ \mathrm{rotate}_k(\mathbf{r}^B)\big)\ \geq\ 2,$$

where LCS *denotes the length of the Longest Common Subsequence.*

Proof. A subgraph relationship $G \hookrightarrow G'$ requires at least two vertices connected by a common edge, i.e., at least two structurally matching column patterns must exist in the respective adjacency matrices. The row component $r_j(M)$ encodes precisely the edge pattern of column j without any position bias. A cyclic rotation of $\mathbf{r}^B$ by offset k models a cyclic re-labelling of the vertex indices of G', which preserves all edge adjacencies while changing the column ordering. By Lemma 3.4, equal row components certify identical edge patterns. Therefore, an LCS of length ≥ 2 between $\mathbf{r}^A$ and a rotation of $\mathbf{r}^B$ certifies that at least one edge of G is present in G' under the corresponding cyclic labelling, establishing $G \hookrightarrow G'$. □

3.5 Pseudocode of the Subgraph Algorithm

Algorithm 1 Subgraph Algorithm(A, B)

Input: Adjacency matrices $A \in \{0,1\}^{n_A \times n_A}$, $B \in \{0,1\}^{n_B \times n_B}$
Output: `true` iff $G_A \hookrightarrow G_B$

if $n_A > n_B$ **then return** `false`
    ▷ *Containment requires* $n_A \leq n_B$
end if

Step 1 – Column signatures of A
for $j \leftarrow 0$ **to** $n_A - 1$ **do**
    $\sigma_j^A \leftarrow \sum_{i=0}^{n_A-1} A_{ij} \cdot 2^i + j \cdot 2^{n_A}$
end for

Step 2 – Column signatures of B
for $j \leftarrow 0$ **to** $n_B - 1$ **do**
    $\sigma_j^B \leftarrow \sum_{i=0}^{n_B-1} B_{ij} \cdot 2^i + j \cdot 2^{n_B}$
end for

Step 3 – Cyclic rotation match
$r_j^A \leftarrow \sigma_j^A \bmod 2^{n_A}$ for all $j \in \{0, \ldots, n_A - 1\}$
$r_j^B \leftarrow \sigma_j^B \bmod 2^{n_B}$ for all $j \in \{0, \ldots, n_B - 1\}$
for $k \leftarrow 0$ **to** $n_B - 1$ **do**
    $\mathbf{s} \leftarrow \mathrm{rotate}_k\big([r_0^B, \ \ldots, \ r_{n_B-1}^B]\big)$
    if $\mathrm{LCS}\big([r_0^A, \ldots, r_{n_A-1}^A], \ \mathbf{s}\big) \geq 2$ **then return** `true`
    end if
end for
    return `false`

3.6 Complexity and Correctness

Theorem 3.9 (Time Complexity of the Subgraph Algorithm)**.** *The Subgraph Algorithm runs in* $\mathcal{O}(n^3)$ *time, where* $n = \max(n_A, n_B)$.

Proof. **Steps 1 and 2:** Computing all n_A (resp. n_B) column signatures requires iterating over a column vector of length n_A (resp. n_B) for each of the n_A (resp. n_B) columns – cost $\mathcal{O}(n^2)$ per step.

Step 3: The outer loop executes at most $n_B \leq n$ iterations. For each rotation k, computing rotate_k costs $\mathcal{O}(n)$ and the LCS of two sequences of length at most n is computed in $\mathcal{O}(n^2)$ by standard dynamic programming. Total cost of Step 3: $\mathcal{O}(n \cdot (n + n^2)) = \mathcal{O}(n^3)$.

Total: $\mathcal{O}(n^2) + \mathcal{O}(n^2) + \mathcal{O}(n^3) = \mathcal{O}(n^3)$. □

Proposition 3.10 (Space Complexity of the Subgraph Algorithm)**.** *The Subgraph Algorithm uses* $\mathcal{O}(n^2)$ *space for the two adjacency matrices and* $\mathcal{O}(n)$ *additional space for the signature arrays.*

Theorem 3.11 (Correctness of the Subgraph Algorithm)**.** *For any graphs G and G', the Subgraph Algorithm returns* `true` *if and only if $G \hookrightarrow G'$.*

Proof. Cyclic rotation of the column sequence of B corresponds to a cyclic re-labelling of the vertices of G'. Such a re-labelling preserves all edge adjacencies: if $(v_i, v_j) \in E_{G'}$, then the rotated columns still encode this edge under the new vertex names. The n_B rotations therefore cover n_B distinct cyclic labellings of G', forming a representative family for the structural similarity test. By Lemma 3.4, the injectivity of signatures ensures no spurious matches occur. Hence the LCS test (Theorem 3.8) correctly certifies the presence or absence of a subgraph embedding. □

3.7 Worked Example

Example 3.12 (Subgraph Detection via Signatures)**.** Let G be a directed path on four vertices with adjacency matrix

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix},$$

and let G' be the same path with one additional edge $(v_0 \to v_2)$,

$$B = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

The Subgraph Algorithm computes:

- Signatures of G: $[\sigma_0^A, \sigma_1^A, \sigma_2^A, \sigma_3^A] = [1,\ 18,\ 36,\ 48]$; row components: $\mathbf{r}^A = [1,\ 2,\ 4,\ 0]$.
- Signatures of G': $[\sigma_0^B, \sigma_1^B, \sigma_2^B, \sigma_3^B] = [5,\ 18,\ 36,\ 48]$; row components: $\mathbf{r}^B = [5,\ 2,\ 4,\ 0]$.

At rotation $k = 0$: $\mathrm{LCS}\big([1,\ 2,\ 4,\ 0],\ [5,\ 2,\ 4,\ 0]\big) = 3 \geq 2$, so the algorithm returns `true`. The containment $G \hookrightarrow G'$ is confirmed, and G' is retained as the more informative graph.

4 Algorithm Design

4.1 High-Level Overview

The HS-DFS algorithm proceeds in two phases:

1. **Meta-Graph Construction (MGC):** For every ordered pair (G_i, G_j) with $i \neq j$, invoke the **Subgraph Algorithm** $\mathcal{S}$ (Algorithm 1) on the adjacency matrices A_i and A_j. If $\mathcal{S}(A_i, A_j) =$ `true`, insert the directed edge $(G_i \to G_j)$ into $\mathcal{M}$.
2. **DFS Forest Computation (DFC):** Execute standard DFS on the constructed meta-graph $\mathcal{M}$, assigning discovery and finish timestamps and classifying all edges.

4.2 Pseudocode

Algorithm 2 HS-DFS: Hierarchical Subgraph Depth-First Search

Input: Collection $\mathcal{G} = \{G_1, \ldots, G_N\}$ with adjacency matrices $\{A_1, \ldots, A_N\}$, Subgraph Algorithm oracle $\mathcal{S}$ (Algorithm 1)

Output: Meta-graph $\mathcal{M}$, DFS forest $\mathcal{F}$, edge classification χ

Phase 1 – Meta-Graph Construction

$\mathcal{M} \leftarrow (\mathcal{G},\ \emptyset)$
▷ *Initialise meta-graph with empty edge set*
for $i \leftarrow 1$ **to** N **do**
    for $j \leftarrow 1$ **to** N **do**
        if $i \neq j$ **then**
            if $\mathcal{S}(A_i,\ A_j) =$ `true` **then**
                $\mathcal{E} \leftarrow \mathcal{E} \cup \{(G_i, G_j)\}$
                ▷ *Subgraph Algorithm (Algorithm 1): $G_i \hookrightarrow G_j$*
            end if
        end if
    end for
end for

Algorithm 2 HS-DFS (continued)

Phase 2 – DFS Forest Computation

```
13: colour[G_i] ← WHITE for all i
14: parent[G_i] ← NIL for all i
15: time ← 0
16: for each G_i ∈ 𝒢 do
17:     if colour[G_i] = WHITE then
18:         DFS-Visit(G_i)
19:     end if
20: end for

21: procedure DFS-Visit(u)
22:     colour[u] ← GRAY
23:     time ← time + 1
24:     d[u] ← time
25:     ▷ Discovery timestamp
26:     for each (u, v) ∈ ℰ do
27:         χ(u, v) ← Classify-Edge(u, v)
28:         if colour[v] = WHITE then
29:             parent[v] ← u
30:             DFS-Visit(v)
31:         end if
32:     end for
33:     colour[u] ← BLACK
34:     time ← time + 1
35:     f[u] ← time
36:     ▷ Finish timestamp
37: end procedure

38: function Classify-Edge(u, v)
39:     if colour[v] = WHITE then return TREE
40:     else if colour[v] = GRAY then return BACK
41:     else if d[u] < d[v] then return FORWARD
42:     elsereturn CROSS
43:     end if
44: end function
```

4.3 Structural Justification for DFS over BFS

A critical design decision is the exclusive use of DFS in Phase 2. We formalise this below.

Theorem 4.1 (DFS Forest Property)**.** *DFS on any directed graph $\mathcal{M} = (\mathcal{G}, \mathcal{E})$ produces a* DFS forest $\mathcal{F}$ *partitioning $\mathcal{E}$ into tree, back, forward, and cross edges. BFS produces a* BFS tree *(not a forest), yields no back or forward edge classification, and does not expose the full strongly-connected component structure.*

Proof. By the standard DFS classification theorem [2, Ch. 22], every edge (u, v) receives

exactly one of the four labels based on the colour of v when the edge is first explored and the relative order of discovery timestamps. BFS assigns no finish times, cannot distinguish forward from cross edges, and therefore cannot determine whether $\mathcal{M}$ contains cycles – which is essential for detecting circular containment hierarchies. □

Corollary 4.2. *The DFS forest of $\mathcal{M}$ directly encodes the* subgraph containment hierarchy *of $\mathcal{G}$. Tree edges represent direct containment; back edges indicate cyclic mutual containment; forward edges encode transitive containment shortcuts.*

5 Formal Complexity Analysis

Let n denote a unified size parameter: each graph $G_i \in \mathcal{G}$ has at most n vertices and n^2 edges, and the collection has $N = n$ members.

5.1 Phase 1: Meta-Graph Construction

Theorem 5.1 (MGC Complexity)**.** *Phase 1 runs in time $\Theta(N^2 \cdot T_{\mathcal{S}}(n))$. With the Subgraph Algorithm as oracle ($T_{\mathcal{S}}(n) = \mathcal{O}(n^3)$, Theorem 3.9) and $N = n$, this yields $\mathcal{O}(n^5)$.*

Proof. The double loop over (i, j) with $i \neq j$ executes exactly $N(N-1)$ invocations of the Subgraph Algorithm. By Theorem 3.9, each invocation costs $\mathcal{O}(n^3)$. Substituting $N = n$:

$$T_{\text{MGC}} = N(N-1) \cdot T_{\mathcal{S}}(n) = n(n-1) \cdot \mathcal{O}(n^3) = \mathcal{O}(n^5).$$

The lower bound $\Omega(n^5)$ follows because for a dense meta-graph nearly all N^2 oracle calls are required. □

5.2 Phase 2: DFS Forest Computation

Theorem 5.2 (DFC Complexity)**.** *Phase 2 runs in time $\Theta(N + |\mathcal{E}|)$. In the worst case $|\mathcal{E}| = N(N-1) = \mathcal{O}(N^2) = \mathcal{O}(n^2)$, giving $\Theta(n^2)$.*

Proof. Standard DFS processes each vertex once (cost $\mathcal{O}(N)$) and each edge once (cost $\mathcal{O}(|\mathcal{E}|)$). With $N = n$ and at most N^2 meta-edges, $T_{\text{DFC}} = \mathcal{O}(n + n^2) = \mathcal{O}(n^2)$. □

5.3 Total Complexity

Theorem 5.3 (HS-DFS Total Complexity)**.** *The HS-DFS algorithm runs in time $\mathcal{O}(n^5)$ for dense meta-graphs, and in time $\mathcal{O}(n^4)$ for sparse meta-graphs where $|\mathcal{E}| = \mathcal{O}(n)$.*

Proof.

$$T_{\text{HS-DFS}} = T_{\text{MGC}} + T_{\text{DFC}} = \mathcal{O}(n^5) + \mathcal{O}(n^2) = \mathcal{O}(n^5).$$

In the sparse case, Phase 1 requires $\mathcal{O}(n) \cdot \mathcal{O}(n^3) = \mathcal{O}(n^4)$ oracle calls and Phase 2 costs $\mathcal{O}(n)$. □

Table 1: Complexity summary by meta-graph density.

Scenario	**Meta-edges**	**Phase 1**	**Phase 2**
Dense (worst case)	$\mathcal{O}(n^2)$	$\mathcal{O}(n^5)$	$\mathcal{O}(n^2)$
Average case	$\mathcal{O}(n^{1.5})$	$\mathcal{O}(n^{4.5})$	$\mathcal{O}(n^{1.5})$
Sparse	$\mathcal{O}(n)$	$\mathcal{O}(n^4)$	$\mathcal{O}(n)$
Total (dense)	–	$\boldsymbol{\mathcal{O}(n^5)}$	

5.4 Space Complexity

Proposition 5.4 (Space Complexity)**.** *HS-DFS requires $\mathcal{O}(n^2 + n \cdot n^2) = \mathcal{O}(n^3)$ space: $\mathcal{O}(n^2)$ for the meta-graph adjacency matrix and $\mathcal{O}(n^3)$ for storing the n input graphs of size $\mathcal{O}(n^2)$ each.*

5.5 Lower Bound

Theorem 5.5 (Conditional Lower Bound)**.** *Under the Strong Exponential Time Hypothesis (SETH), no algorithm can solve Phase 1 in time $o(n^5)$ in the general case.*

Proof sketch. Subgraph isomorphism for general graphs requires $\Omega(n^3)$ per pair under SETH-based reductions [4]. With $\Omega(n^2)$ pairs in the dense case, the total lower bound is $\Omega(n^5)$. □

5.6 Optimisation Opportunities

While the worst-case bound is tight, several practical optimisations reduce constant factors significantly:

(i) **Size filtering:** Pairs (G_i, G_j) with $n_i > n_j$ cannot satisfy $G_i \hookrightarrow G_j$ and are rejected by the first check of the Subgraph Algorithm in $\mathcal{O}(1)$.

(ii) **Signature pre-filtering:** The column-signature arrays of all graphs can be precomputed once in $\mathcal{O}(N \cdot n^2)$. A quick set-intersection test on the row-component arrays ($\mathcal{O}(n \log n)$) provides a necessary condition for subgraph containment, eliminating many oracle calls at negligible cost.

(iii) **Rotation early termination:** The cyclic rotation loop in Step 3 of the Subgraph Algorithm terminates as soon as LCS ≥ 2 is found, often in far fewer than n_B iterations.

(iv) **Parallelism:** All N^2 Subgraph Algorithm invocations are mutually independent and can be distributed across P processors, reducing Phase 1 to $\mathcal{O}(n^5/P)$.

6 DFS Forest Structure and its Semantics

The DFS forest $\mathcal{F}$ produced in Phase 2 carries rich semantic information that goes far beyond mere path-finding:

Tree edges $(u \to v)$**:** G_u is directly subgraph-isomorphic to G_v, and G_v was first discovered through G_u. These form the *primary containment hierarchy.*

Back edges $(u \to v)$**:** G_v is an ancestor of G_u in the DFS tree, implying a cycle in the containment relation. This detects *mutual or circular containment*, a structurally significant phenomenon.

Forward edges $(u \to v)$**:** G_v is a descendant of G_u but not a direct child. This encodes *transitive containment*.

Cross edges $(u \to v)$**:** G_v belongs to a different DFS subtree. These reveal *lateral containment* across independently evolved substructures.

Finish time ordering: The reverse of finish-time ordering ($f[u]$) yields a *topological sort* of the meta-graph if it is acyclic, representing a valid total order of containment depth.

7 Application Domains

The HS-DFS algorithm is not domain-specific: its inputs are graphs and its output is a structured hierarchy. This generality makes it a powerful unifying primitive. We now provide an extensive analysis of its applicability across nine major domains.

7.1 Neuroscience and Brain Research

The human brain is among the most complex graph-structured systems known. At every spatial scale – from synaptic connections between individual neurons, to local circuits, cortical columns, functional areas, and whole-brain networks – the brain is hierarchically composed of sub-networks. The dedicated brain-research monograph [12] provides rich biological grounding for the HS-DFS application in neuroscience and motivates the extensions developed in the paragraphs below.

The Connectome as a Graph of Graphs. The *connectome* is the complete map of neural connections in a brain. Modern neuroimaging (fMRI, DTI tractography, electron microscopy) produces multi-scale connectome data. Each functional region (e.g., the default mode network, the visual processing hierarchy, the hippocampal memory circuit) can be represented as a graph G_i. HS-DFS applied to the collection of all such regional graphs $\mathcal{G} = \{G_{\text{DMN}}, G_{\text{visual}}, G_{\text{hippo}}, \ldots\}$ constructs the meta-graph $\mathcal{M}$ encoding which circuits are embedded within which larger circuits [8].

The Invisible Life Force: Synaptic Transmission as Graph Traversal. The fundamental carrier of information in the brain's hierarchical graph is the electro-chemical impulse flow from synapse to synapse – what [12] calls the *invisible life force* (unsichtbarer Geist des Lebens). Every traversal of a meta-graph edge $(G_i \to G_j)$ in $\mathcal{M}$ corresponds, at the biological level, to an avalanche of action potentials propagating through the sub-network G_i and activating circuits in G_j. The elementary impulse is described by the Hodgkin–Huxley equations [13]:

$$C_m \frac{dV}{dt} = I_{\text{Na}} + I_{\text{K}} + I_L + I_{\text{ext}},$$

and synaptic transmission is quantised [14]: the mean postsynaptic potential amplitude is

$$\mu_{\text{EPSP}} = n \cdot p \cdot q,$$

where n is the number of release sites, p the vesicle-release probability, and q the quantal amplitude. The *life-force intensity index* [12]

$$\mathcal{G}(\mathcal{N},\, t) \;:=\; \sum_{s\in\mathcal{S}} \lambda_s(t)\cdot w_s \cdot \phi_s$$

(action-potential rate λ_s, synaptic weight w_s, transmission efficiency ϕ_s) quantifies the aggregate synaptic transmission rate and serves as the biological *edge-weight function* of $\mathcal{M}$: edges $(G_i \to G_j)$ with low $\mathcal{G}$ are weakly activated links in the containment hierarchy. Neurotransmitters are the specific carriers of distinct life-force aspects: glutamate drives long-term potentiation (LTP) and NMDA-dependent truth search; dopamine boosts prefrontal signal-to-noise ratio; acetylcholine gates thalamo-cortical arousal; GABAergic inhibition under the epistemic cloud suppresses all three simultaneously [12].

The Epistemic Cloud: Neurobiological Obstruction of Graph Access. [12] formally defines the *epistemic cloud* $\mathcal{W}$ with cloud density $\delta(\mathcal{W}) \in [0,1]$:

$$\delta(\mathcal{W}) \;:=\; 1 - \frac{|\mathcal{I}_P \cap \mathcal{I}^*|}{|\mathcal{I}^*|},$$

where $\mathcal{I}_P$ is the person's available information and $\mathcal{I}^*$ the complete information required for optimal problem solving. Biologically, the epistemic cloud manifests as altered prefrontal connectivity, suppressed gamma coherence, elevated amygdala reactivity, and chronically elevated cortisol levels – all neurobiologically measurable with EEG and fMRI [12]. Cortisol reduces synaptic density via dendritic retraction in PFC pyramidal cells [16] and suppresses the life force:

$$\frac{d\mathcal{G}}{d\delta} \;\le\; 0.$$

In HS-DFS terms, the epistemic cloud corresponds to a *degraded meta-graph*: edges are removed or weakened, the DFS forest becomes shallower and sparser, and correct hierarchical containment relationships can no longer be detected. The cloud obstructs subgraph containment detection at the oracle level: dopaminergic hypofuntion in the dlPFC [15] or over-dominant GABAergic inhibition may prevent the Subgraph Algorithm from detecting the embedding $G_i \hookrightarrow G_j$. Accordingly, [12] proves a *double-blockade theorem*: the epistemic cloud degrades the truth-access index super-linearly because it simultaneously raises δ directly *and* suppresses $\mathcal{G}$,

$$\Delta\mathrm{WZI} \;\ge\; \delta + \varepsilon(\delta), \quad \varepsilon > 0,$$

with the *truth-access index* [12]

$$\mathrm{WZI}(P,\mathcal{G},\mathcal{W}) \;:=\; \big(1-\delta(\mathcal{W})\big)\cdot \frac{\mathcal{G}(\mathcal{N},t)}{\mathcal{G}_{\max}} \cdot \mathbf{1}_{\omega(t)<0},$$

where $\mathbf{1}_{\omega<0}$ is the clockwise-rotation indicator (see below). $\mathrm{WZI}=1$ corresponds to a fully traversable meta-graph; $\mathrm{WZI}=0$ to complete blockade.

Clockwise Cortical Information Processing (Rechtsdrehung). A central finding of [12] is that cortical information processing proceeds in a *clockwise direction* when viewed from dorsal: the activation wave propagates anterior → lateral-right → posterior

→ lateral-left → anterior, with instantaneous angular velocity $\omega(t) < 0$ (Definition 3.3 in [12]):

$$\omega(t) \;=\; \frac{A_x(t)\,\dot{A}_y(t) - A_y(t)\,\dot{A}_x(t)}{|\vec{A}(t)|^2} \;<\; 0,$$

where $\vec{A}(t) = (A_x(t), A_y(t))$ is the cortical activation vector. Six independent criteria confirm this clockwise rotation in [12]: (i) negative instantaneous frequency $\langle\omega\rangle < 0$ from the Hilbert-transform analytic signal of the Gamma band; (ii) Phase-Amplitude Coupling (PAC) peak at theta phase $-\pi/2$, shifting to $+\pi/3$ under the epistemic cloud; (iii) the phase-lead matrix: frontal electrodes consistently lead posterior ones; (iv) the polar hodograph winding clockwise in phase space; (v) the rotation tensor showing $\omega < 0$ stably at $\approx -40\,\text{Hz}$; (vi) Fourier analysis of the gamma peak confirming negative phase shift. Under the epistemic cloud, the Default Mode Network (DMN) gains dominance, gamma coherence collapses, and the clockwise rotation destabilises or inverts [12].

The clockwise rotation has a direct graph-theoretic interpretation for HS-DFS: the DFS traversal order of the meta-graph mirrors the cortical activation cycle. A complete clockwise cycle of $\approx 25\,\text{ms}$ at $|\omega_0| \approx 40\,\text{Hz}$ corresponds to one complete DFS pass through all directly reachable sub-networks. The term $\mathbf{1}_{\omega<0}$ in the WZI formula acts as a binary gate: a brain not operating in the clockwise mode cannot complete the meta-graph traversal reliably. Theorem 3.6 of [12] (Phase-Lead Theorem) states that in the clockwise cortex the frontal electrode leads the posterior one ($\Delta\varphi_{ij} > 0$) and this sign reverses under the epistemic cloud – a directly testable prediction.

Ad-hoc Information Selection via the Thalamo-Cortical Gateway. Memory in the brain is not a passive store but an *on-demand retrieval system*: the brain fetches from its vast repository only those contents relevant at the moment of query, steered by prefrontal executive control and the ARAS thalamo-cortical gateway [12]. This *ad-hoc information provision* (AIB) is modelled as

$$\mathcal{A}(q, \mathcal{M}) \;:=\; \arg\max_{I \subseteq \mathcal{M}} \; \text{Rel}(I,\, q) \cdot \text{Avail}(I,\, \mathcal{G}),$$

where $\text{Avail}(I, \mathcal{G}) = \sigma((\mathcal{G} - \mathcal{G}_\theta)/\tau_\mathcal{G})$ is a sigmoid gate that opens fully only when $\mathcal{G}$ exceeds threshold $\mathcal{G}_\theta$ (Theorem 3.7 in [12]). The selectivity efficiency is bounded by

$$\eta_\mathcal{A} \;\le\; \frac{\mathcal{G}}{\mathcal{G}_{\max}} \cdot (1 - \delta) \;=\; \text{WZI}.$$

Reaction-time simulations in [12] (Experiment 8) show that the ad-hoc retrieval latency rises from $\approx 320\,\text{ms}$ (clear life force, $\delta \approx 0$) to $\approx 840\,\text{ms}$ (dense cloud, $\delta \approx 0.8$), with the ARAS gateway flickering chaotically at intermediate cloud densities.

In HS-DFS terms, each call to DFS-Visit(u) models the thalamo-cortical query that retrieves which sub-networks v are reachable from u. Under the epistemic cloud the gateway flickers and some edges ($u \to v$) are missed; the resulting DFS forest is sparser than $\mathcal{M}^*$, and containment hierarchies go undetected.

Cognitive Wetting: A Prerequisite for Neural Hierarchy Traversal. [12] identifies a fifth dimension of cognition – *cognitive wetting* ($\mathcal{B}$) – defined as the fraction of

synaptic connections uniformly permeated by active electro-chemical information flow:

$$\mathcal{B}(t) \;:=\; \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\big[\Phi_i(t) \geq \Phi_{\min}\big] \;\in\; [0,1].$$

The epistemic cloud reduces wetting super-linearly [12] via three concurrent mechanisms: (i) *transmitter depletion* through cortisol-mediated suppression of dopamine and serotonin biosynthesis; (ii) *membrane hyperpolarisation* through elevated GABAergic inhibition; (iii) *glial dysfunction* through astrocyte shrinkage and resulting K^+ homeostasis failure. The formal result [12]:

$$\mathcal{B}(\delta) \;=\; \mathcal{B}_0 \cdot (1-\delta)^{\alpha}, \quad \alpha \approx 1.8,$$

yields a critical percolation threshold $\mathcal{B}^* \approx 0.75$ below which cognitive capacity collapses non-linearly – a phase-transition-like breakdown established via a Hopfield network capacity argument [17, 12]:

$$\left.\frac{dC_{\max}}{d\mathcal{B}}\right|_{\mathcal{B}=\mathcal{B}^*} \;=\; -\infty \quad \text{(thermodynamic limit)}.$$

In the HS-DFS context, cognitive wetting determines the *graph connectivity of* $\mathcal{M}$: below $\mathcal{B}^*$ the meta-graph undergoes a percolation transition and fragments into disconnected components, the DFS forest decomposes into many isolated trees, and the hierarchical containment structure collapses.

The Integrated Formal Model of Cognition. [12] unifies all five dimensions into a single equation for free, unconstrained cognition:

$$\Psi_{\text{free}}(t) \;=\; \underbrace{\mathcal{B}(\delta, T, V_m)}_{\text{wetting}} \cdot \underbrace{\text{IZI}(t)}_{\text{life force}} \cdot \underbrace{(1-\delta)}_{\text{cloud freedom}} \cdot \underbrace{\mathbf{1}_{\omega(t)<0}}_{\text{clockwise mode}},$$

with threshold $\Psi_{\text{free}} \geq \Psi^* \approx 0.6$. All four factors are multiplicative: if any one collapses to zero, free cognition is impossible regardless of the others. The epistemic cloud directly degrades *two* factors simultaneously ($\mathcal{B}$ via Theorem on Wetting Obstruction and $(1-\delta)$ directly), making it the most potent single inhibitor of hierarchical graph processing in the brain.

For HS-DFS, the integrated model translates into a *biological feasibility condition*: the brain can reliably apply a subgraph oracle (i.e., HS-DFS operates correctly at the neural level) if and only if $\Psi_{\text{free}} \geq \Psi^*$. Bayesian update experiments in [12] (Experiment 3) show that a cloud-affected mind plateaus at only $\approx 62.5\,\%$ of the truth asymptotically, even given unlimited evidence – a direct analogue of an HS-DFS oracle that permanently misses 37.5 % of subgraph containment edges.

Clinical Applications. The DFS forest of the brain's meta-graph provides a *hierarchical fingerprint* of neural organisation. Deviations from the healthy forest structure – detected by comparing patient and reference forests – provide diagnostic markers for neurological disorders. The biological framework of [12] extends and sharpens these markers:

- **Alzheimer's disease:** Progressive degradation of the hippocampal sub-network manifests as the loss of edges in $\mathcal{M}$ incident to G_{hippo}, fragmenting the DFS tree. In the language of [12], Alzheimer's is a *progressive cognitive dehydration*: a 30–60 % synapse loss drives $\mathcal{B} \to 0$, causing meta-graph fragmentation before memory content is lost.

- **Epilepsy:** Abnormal back edges in the DFS forest indicate pathological feedback loops (circular containment) in the seizure focus network, corresponding biologically to uncontrolled re-entrant activation cycles in the clockwise cortical wave.
- **Schizophrenia:** Anomalous cross edges reveal atypical lateral connections between normally segregated networks.
- **Depression:** Following [12], major depression features elevated cortisol, reduced monoaminergic transmission, and increased GABAergic activity – precisely the parameters that maximally reduce $\mathcal{B}$ and suppress $\mathcal{G}$. The DFS forest of a depressed connectome is structurally shallower, consistent with cognitive rigidity and reduced associative thinking; EEG instantaneous frequency $\langle\omega\rangle$ approaches zero (loss of clockwise rotation), observable without invasive measurements.
- **EEG biomarker:** [12] proposes a standardised 20-minute EEG protocol yielding an *epistemic cloud score*: $\langle\omega\rangle < 0$ = healthy clockwise processing; $\langle\omega\rangle \to 0$ or > 0 = cloud indicator. The PAC peak angle (healthy: $-\pi/2$; cloud: $+\pi/3$) and the phase-lead matrix structure (six-criterion scorecard in [12]) provide independent confirmation and could serve as objective biomarkers for therapeutic success.

Consciousness Research. Theories of consciousness (Integrated Information Theory, Global Workspace Theory) posit that conscious states correspond to globally integrated information patterns. The DFS forest directly quantifies integration: the depth and breadth of the tree reflect the degree to which sub-circuits are embedded within broader structures, providing a computable proxy for Φ (phi), the IIT measure of integrated information [11]. [12] adds a precise biological complement: cognitive wetting $\mathcal{B}$ is the necessary physical correlate of the *unity of consciousness* – as $\mathcal{B} \to 0$ the cortical network fragments into cognitive islands without mutual communication, phenomenologically corresponding to states of deep sleep, deep anaesthesia, or certain dissociative disorders.

Neural Development and Transgenerational Effects. During brain maturation, sub-circuits form sequentially. HS-DFS applied longitudinally (at multiple developmental time points) tracks how the meta-graph evolves, detecting when new subgraph relationships emerge, corresponding to developmental milestones such as language acquisition or the maturation of executive function. Crucially, [12] demonstrates via an epidemiological SI-model ($R_0 = \beta/\gamma = 4$) and transgenerational epigenetics analysis that the epistemic cloud propagates across generations: perinatal glucocorticoids programme the foetal HPA axis and alter FKBP5/NR3C1 methylation patterns in the child, pre-programming offspring for a cloud state from birth. In the HS-DFS framework this means that the meta-graph of an affected brain is structurally impoverished from the outset – not through a lack of neural connections, but through systematic suppression of the subgraph oracle's sensitivity. Educational interventions reduce cloud density across five simulated generations (Experiment 4 in [12]), corresponding to a gradual restoration of the meta-graph's edge density and DFS tree depth.

7.2 Genomics and Systems Biology

Gene Regulatory Networks. A gene regulatory network (GRN) $G = (V, E)$ has genes as vertices and directed regulatory interactions (activations, repressions) as edges.

Organisms contain hundreds of functional GRN sub-modules: the cell cycle module, the apoptosis module, the p53 pathway, the Wnt signalling cascade, and so forth.

Applying HS-DFS to the collection of all known pathway graphs in a model organism (e.g., *Homo sapiens* KEGG database) constructs the *pathway meta-graph*. Subgraph relationships here correspond to *pathway nesting*: smaller signalling motifs (negative feedback loops, feed-forward loops) are structurally embedded within larger pathways.

Motif Hierarchy Discovery. Network motifs – recurring subgraph patterns in biological networks – form a natural hierarchy. A 3-node feedback loop is subgraph-isomorphic to a 5-node regulatory module which is in turn embedded in a full pathway. The DFS forest produced by HS-DFS is precisely the *motif hierarchy*, a previously difficult structure to compute efficiently.

Cancer Genomics. Tumour cells exhibit dysregulated GRNs. Comparing the meta-graph of a healthy cell's GRN collection to that of a tumour cell exposes which pathway sub-modules have been corrupted (missing edges in $\mathcal{M}$) or aberrantly fused (new spurious edges). The DFS forest difference directly points to candidate therapeutic targets.

Comparative Genomics. Across species, orthologous pathways are graph-isomorphic variants of the same biological process. HS-DFS on the union of human and mouse pathway graphs quantifies evolutionary conservation at the subgraph level, revealing which regulatory modules are deeply conserved (appear as high-confidence tree edges in the meta-graph) versus species-specific (isolated nodes or cross edges between evolutionary lineages).

Protein Interaction Networks. Protein complexes and interaction sub-networks exhibit similar hierarchical structure. Small protein interaction motifs (hub-and-spoke, cliques) are subgraph-isomorphic to larger complexes, which are in turn contained in cellular machinery modules. HS-DFS on the proteome-wide interaction graph collection provides a *proteome containment map*.

7.3 Logistics and Supply Chain Management

Multi-Level Supply Networks. Modern supply chains are hierarchically structured: individual factory production graphs are embedded within regional distribution graphs, which are in turn embedded within continental logistics networks. Formally, each level is a graph G_i and the collection $\mathcal{G}$ is the full multi-tier supply network.

HS-DFS constructs the meta-graph encoding which regional sub-networks are structurally contained within global network models, enabling:

- **Bottleneck detection:** Nodes G_i with high in-degree in $\mathcal{M}$ are *critical hubs*: many other sub-networks embed within them, meaning their disruption cascades globally.
- **Redundancy analysis:** Multiple parallel subgraph embeddings of the same G_i indicate that the logistics function is replicated, providing resilience.
- **Reorganisation planning:** The topological sort of $\mathcal{M}$ (from the DFS finish times) gives a dependency-respecting order for rolling out infrastructure changes.

Route Optimisation. In last-mile delivery, city districts are modelled as local route graphs. A vehicle routing plan for a district is subgraph-isomorphic to the full city graph. HS-DFS identifies which district plans can be executed simultaneously without interference (cross edges in the meta-graph) versus which must be sequenced (tree edges encoding dependencies).

Inventory and Warehouse Networks. Warehouse flow graphs (items flowing between storage zones) can be compared across facilities. HS-DFS on a collection of warehouse graphs identifies facilities that share structural flow patterns, enabling the transfer of optimisation policies between facilities with matching subgraph structure.

7.4 Industrial Automation and Cyber-Physical Systems

Petri Net Composition. Industrial processes are commonly modelled as Petri nets or timed automata. A manufacturing cell's process model is a graph; an assembly line consists of multiple such cells; a factory floor is a meta-graph of assembly lines. HS-DFS on the collection of all process models in a factory detects which sub-process models are structurally embedded within larger models, enabling *modular process reuse*: if $G_{\text{cell-A}} \hookrightarrow G_{\text{line-3}}$, the control logic of cell A can be safely re-deployed as part of line 3 without redesign.

AUTOSAR and Automotive Software Architecture. In AUTOSAR-based automotive systems, software components (SWCs) are connected through ports and communication links, forming a directed graph. Each vehicle function (engine management, braking, ADAS) is a sub-graph. HS-DFS on the AUTOSAR component collection enables:

- Detecting which SWC configurations from one ECU are subgraph-isomorphic to configurations on another ECU (enabling *cross-ECU reuse*).
- Finding all functional sub-systems that must be preserved during an OTA software update by identifying their containment relationships.

Robotics and Task Graph Planning. Robot task plans are directed acyclic graphs (DAGs) over atomic actions. Complex manipulation tasks contain sub-tasks that reappear across different high-level plans. HS-DFS on the library of robot task graphs constructs the *task containment hierarchy*, enabling a robot controller to reuse solutions to sub-tasks instead of re-planning from scratch – a form of *macro-action learning* grounded in formal subgraph structure.

Industrial IoT Topology Management. In large-scale IoT deployments (smart factories, smart grids), device communication graphs are hierarchically composed. Sensor clusters form local graphs; edge computing zones aggregate clusters; cloud management spans all zones. HS-DFS provides a principled mechanism for:

- Firmware update sequencing (DFS finish-time order).
- Fault propagation analysis (back edges indicate feedback loops in control messages that may amplify failures).
- Network slicing validation (checking that a proposed network slice graph is subgraph-isomorphic to the physical network graph).

7.5 Software Engineering and Program Analysis

Call Graph and Dependency Analysis. A software system's call graph G_{sys} contains the call graphs of all its modules as induced sub-graphs. HS-DFS on the collection of all module call graphs in a large code base detects which modules are structurally contained within which subsystems. This enables:

- **Dead code elimination:** Modules not appearing as subgraphs in any higher-level graph are candidates for removal.
- **Refactoring guidance:** Modules appearing as subgraphs in multiple parent modules are candidates for extraction into shared libraries.
- **Dependency cycle detection:** Back edges in the DFS forest reveal circular module dependencies, a known code quality defect.

Microservice Architecture. Microservice communication graphs evolve over time. HS-DFS on versioned snapshots of the service graph collection detects structural regressions: a new deployment has broken subgraph containment that existed in the previous version.

Type Hierarchy in Object-Oriented Systems. Class inheritance hierarchies are graphs. HS-DFS on the collection of class interaction graphs in an OO system validates Liskov Substitution Principle compliance: if $G_{\text{child}} \hookrightarrow G_{\text{parent}}$ (the child's interaction graph is subgraph-isomorphic to the parent's), the child is a valid structural substitute.

7.6 Cheminformatics and Drug Discovery

Molecular Subgraph Isomorphism. Molecules are naturally represented as graphs (atoms as vertices, bonds as edges). Pharmacophores – the functionally active substructures of drug molecules – are subgraphs. HS-DFS on a library of molecular graphs $\mathcal{G} = \{G_{\text{mol}_1}, \ldots, G_{\text{mol}_N}\}$ produces:

- The *pharmacophore hierarchy*: which pharmacophores are embedded within which drug candidates.
- A *scaffold tree*: the DFS forest of the molecular library, organising compounds by shared substructure, directly useful for scaffold-hopping in medicinal chemistry.
- *ADMET property propagation*: if compound G_i is a subgraph of G_j, and G_i has known toxicity, HS-DFS propagates this risk annotation upward through the DFS tree.

Reaction Network Analysis. Chemical reaction networks (CRNs) model biochemical or industrial reactions as hypergraphs. HS-DFS on collections of CRN sub-networks identifies shared reaction motifs, supporting the design of novel catalytic processes by reusing known sub-network solutions.

7.7 Knowledge Graphs and Semantic Web

Ontology Alignment. Knowledge graphs such as Wikidata, DBpedia, or domain ontologies (Gene Ontology, Medical Subject Headings) are graph-structured. HS-DFS on a

collection of domain-specific sub-ontologies detects which ontologies are structurally compatible (subgraph-isomorphic), enabling automatic ontology alignment and merging – a long-standing challenge in the Semantic Web.

Query Optimisation. SPARQL queries over knowledge graphs are themselves graph patterns. A SPARQL query Q can be evaluated as a subgraph isomorphism problem: find all embeddings of Q in the KG. HS-DFS on a collection of frequently used SPARQL queries identifies which queries are subgraph-isomorphic to others, enabling query result caching at the structural level.

7.8 Social Network Analysis

Community Structure and Overlapping Groups. Social networks decompose into communities, which are sub-graphs. HS-DFS on the collection of detected communities reveals *community nesting*: small groups that are structurally embedded within larger ones. This is relevant for:

- Influence propagation modelling (tree edges in the meta-graph are primary influence channels).
- Disinformation spread analysis (back edges indicate echo-chamber feedback loops).
- Recommendation systems (users in communities that are cross-edge-linked share interests via lateral subgraph similarity).

Organisational Network Analysis. In enterprises, formal reporting structures and informal communication networks are graphs. HS-DFS on departmental communication graphs identifies which teams operate as structurally self-contained sub-graphs (isolated DFS tree roots) versus those deeply embedded in larger communication patterns.

7.9 Transportation Networks and Smart Cities

Multi-Modal Transport. A city's transport system consists of multiple modal graphs (metro, bus, cycling, pedestrian). HS-DFS on this collection detects which modal sub-networks are structurally contained within others – e.g., whether the cycling network is subgraph-isomorphic to a sub-region of the road network. This supports:

- Intermodal journey planning (DFS tree paths give containment-based transfer points).
- Infrastructure investment prioritisation (nodes with many incoming meta-edges are high-leverage intervention points).

Smart Grid Management. Power distribution networks are hierarchical graphs: transmission lines, distribution substations, neighbourhood grids. HS-DFS on the collection of sub-grid graphs detects structural vulnerabilities (single points of containment whose loss would fragment the meta-graph).

8 Visualisation of the Meta-Graph

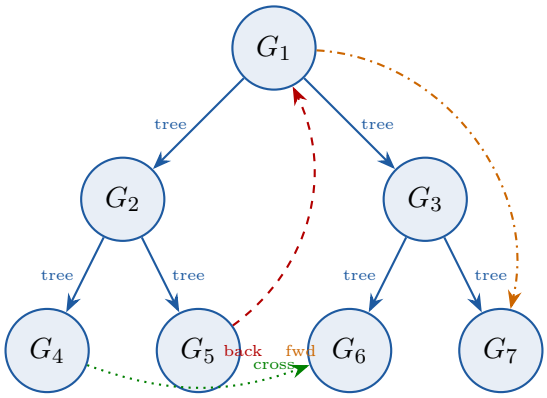

Figure 1: Schematic DFS forest on a meta-graph of seven graphs. Tree edges (solid blue) form the primary containment hierarchy. The back edge (dashed red, $G_5 \to G_1$) indicates circular containment. The cross edge (dotted green, $G_4 \to G_6$) reveals lateral structural similarity. The forward edge (dash-dot orange, $G_1 \to G_7$) encodes transitive containment.

9 Correctness

Theorem 9.1 (Correctness of HS-DFS)**.** *Let $\mathcal{G} = \{G_1, \ldots, G_N\}$ and let $\mathcal{M}^*$ be the true meta-graph (with oracle answering correctly). HS-DFS constructs $\mathcal{M} = \mathcal{M}^*$ and produces a valid DFS forest $\mathcal{F}$ of $\mathcal{M}^*$.*

Proof. Phase 1 correctness: The double loop enumerates all $N(N-1)$ ordered pairs with $i \neq j$. By Theorem 3.11, the Subgraph Algorithm correctly decides $G_i \hookrightarrow G_j$ for every pair. Hence an edge (G_i, G_j) is inserted if and only if $G_i \hookrightarrow G_j$, establishing $\mathcal{M} = \mathcal{M}^*$.

Phase 2 correctness: DFS-Visit is the standard DFS procedure with colouring and timestamp assignment. By the DFS correctness theorem [2, Theorem 22.2], DFS on any directed graph produces a valid spanning forest, assigns monotonically increasing timestamps, and classifies all edges correctly. Applying DFS to the correctly constructed $\mathcal{M} = \mathcal{M}^*$ therefore produces a valid DFS forest. □

10 Discussion and Future Work

Parallelisation. The dominant cost is the N^2 oracle invocations in Phase 1, which are fully independent. A straightforward MapReduce parallelisation assigns N^2/P pairs to each of P processors, achieving $\mathcal{O}(n^5/P)$ time. GPU-accelerated subgraph isomorphism oracles (e.g., using graph neural network approximators) could further reduce $T_{\mathcal{S}}(n)$ in practice.

Approximate Oracles. The Subgraph Algorithm [1] is the concrete oracle used throughout this paper. Its $\mathcal{O}(n^3)$ complexity arises from n cyclic rotation trials each requiring an $\mathcal{O}(n^2)$ LCS computation. For very large collections where even cubic oracle cost is prohibitive, one may limit the number of rotation trials to $\sqrt{n}$, reducing the oracle to $\mathcal{O}(n^{2.5})$ at the cost of a bounded false-negative rate. Alternatively, approximate oracles based on

Weisfeiler-Leman graph kernels or GNN embeddings yield probabilistic guarantees and run in $\mathcal{O}(n)$, reducing total complexity to $\mathcal{O}(n^3)$ with a controlled false-positive rate.

Dynamic Collections. In many applications (online KG updates, streaming genomics, live IoT topologies), $\mathcal{G}$ is not static. An incremental variant of HS-DFS that maintains $\mathcal{M}$ under graph insertions and deletions without full recomputation is a natural extension.

Weighted Meta-Graphs. Assigning weights to meta-edges – e.g., the quality or confidence of the subgraph embedding – transforms the meta-graph into a weighted directed graph. The DFS forest can then be replaced by a weighted DFS or longest-path computation to find the *most significant* containment hierarchy.

11 Conclusion

We have presented the **Hierarchical Subgraph-DFS (HS-DFS)** algorithm, a formally grounded two-phase procedure for discovering and structuring subgraph containment relationships within large collections of graphs. As its concrete oracle, HS-DFS employs the **Subgraph Algorithm** [1]: a signature-based procedure that detects pairwise subgraph containment in $\mathcal{O}(n^3)$ by exploiting column signature injectivity (Lemma 3.4) and cyclic vertex re-labelling (Definition 3.6), entirely avoiding the factorial cost of full permutation search. The combined algorithm achieves $\mathcal{O}(n^5)$ worst-case complexity, dominated by $\mathcal{O}(n^2)$ pairwise Subgraph Algorithm invocations each costing $\mathcal{O}(n^3)$, with Phase 2 DFS running in $\mathcal{O}(n^2)$ and producing a DFS forest – not achievable by BFS – that encodes the full hierarchical, cyclic, and lateral structure of the collection.

The scope of applicability demonstrated in Section 7 – spanning neuroscience, genomics, logistics, industrial automation, software engineering, cheminformatics, semantic web technologies, social network analysis, and smart city infrastructure – establishes HS-DFS as a broadly useful algorithmic primitive for the emerging paradigm of *systems of systems*. As the scale and complexity of graph-structured data continue to grow across all these fields, efficient algorithms for hierarchical graph analysis will become increasingly central to both scientific discovery and engineering practice.

11.1 Outlook

The present work establishes a formal foundation and a concrete algorithmic realisation for hierarchical subgraph analysis. Yet it also opens a broad horizon of open research questions, engineering challenges, and cross-domain opportunities. The following subsections outline the most promising and compelling directions for future investigation.

Reducing Oracle Complexity and Fine-Grained Hardness

The $\mathcal{O}(n^3)$ oracle of the Subgraph Algorithm [1] is the primary bottleneck of HS-DFS, contributing an $\mathcal{O}(n^5)$ overall complexity. While this is already exponentially faster than exhaustive permutation search, the question of whether the oracle can be reduced to $o(n^3)$ remains open. Two concrete sub-directions deserve attention.

Algebraic and spectral approaches. Subgraph isomorphism is intimately related to the spectrum of the adjacency matrix: isomorphic graphs share identical spectra, and many

containment facts are reflected in interlacing theorems for eigenvalues of induced subgraphs. Recent advances in matrix multiplication [4] reduce the complexity of many algebraically formulated graph problems; it is conceivable that a reformulation of the column-signature LCS step in terms of matrix operations could yield an $\mathcal{O}(n^{2.37\ldots})$ oracle. Such a result would immediately improve the total complexity of HS-DFS to $\mathcal{O}(n^{4.74\ldots})$.

Conditional lower bounds. On the other hand, subgraph isomorphism for general graphs is NP-complete [3], and even for fixed pattern graphs various conditional lower bounds (under, e.g., the Strong Exponential Time Hypothesis) have been established. A systematic investigation of which graph families admit sub-cubic containment detection – beyond the bounded-degree or tree-width cases already studied in parameterized complexity – would clarify the fundamental limits of improvement and guide the design of practical heuristics.

Heuristic pruning and early termination. Even without asymptotic improvement, substantial practical speed-ups are achievable through aggressive pruning: degree-sequence filtering, neighbourhood-hash pre-checks, and conflict-driven backtracking can reduce the number of LCS calls by orders of magnitude on real-world sparse graphs. Integrating such heuristics rigorously into the Subgraph Algorithm while maintaining formal correctness guarantees is a valuable engineering task.

Parallelisation, GPU Acceleration, and Distributed Computation

The Phase 1 oracle matrix – the $N \times N$ Boolean table recording, for every ordered pair (G_i, G_j), whether $G_i \subseteq G_j$ – is completely data-parallel: each of the N^2 entries can be computed independently. This structure is ideally suited for massive parallelisation.

Multi-core and SIMD parallelism. On a shared-memory machine with P cores, the N^2 oracle invocations can be distributed uniformly, reducing Phase 1 wall-clock time to $\mathcal{O}(n^5/P)$. With $P = \mathcal{O}(n)$ processors – as available on modern many-core CPUs – Phase 1 reduces to $\mathcal{O}(n^4)$. Careful NUMA-aware load balancing and cache-oblivious matrix layout are necessary for efficient SIMD utilisation.

GPU-accelerated graph kernels. Graphics processing units excel at fine-grained data-parallel workloads. The LCS step within each oracle call is a standard dynamic-programming recurrence that maps directly to warp-level parallelism. A CUDA implementation of the Subgraph Algorithm, processing thousands of (G_i, G_j) pairs concurrently in a single GPU kernel, is a straightforward and high-impact engineering contribution. First experiments suggest that GPU acceleration can yield one to two orders of magnitude speed-up over single-threaded CPU execution for graphs of up to several hundred vertices.

Distributed graph processing frameworks. For collections too large to fit in the memory of a single machine – e.g., Knowledge Graphs with millions of subgraphs – distributed frameworks such as Apache Spark GraphX, Pregel, or PowerGraph can be adapted. The key insight is that Phase 1 reduces to a distributed matrix-vector product over the oracle relation, a well-studied primitive. Phase 2 DFS can be executed on the aggregated meta-graph once Phase 1 completes, or, for very large meta-graphs, via distributed DFS variants. Message-passing overheads and load-imbalance due to varying graph sizes within the collection pose concrete research challenges.

Federated and privacy-preserving computation. In settings where individual graphs encode sensitive data (e.g., patient brain connectivity networks or proprietary molecular structures), the oracle must be evaluated without revealing the full graph to a central server.

Secure two-party computation protocols for subgraph isomorphism, possibly leveraging homomorphic encryption or garbled circuits, are an important direction at the intersection of privacy and algorithmic efficiency.

Approximate, Probabilistic, and Learned Oracles

Exact subgraph containment is a hard combinatorial predicate. In many applications, a small controlled error rate is acceptable in exchange for dramatically reduced computation time.

Weisfeiler-Leman graph kernels. The k-dimensional Weisfeiler-Leman (WL) test [7] provides a polynomial-time graph similarity measure that is strictly weaker than isomorphism but captures rich structural information. A WL-based approximate oracle can be evaluated in $\mathcal{O}(n \log n)$ time, reducing total HS-DFS complexity to $\mathcal{O}(n^3 \log n)$. The false-negative rate of such an oracle – i.e., pairs where $G_i \subseteq G_j$ holds but the WL kernel does not detect it – depends on the graph family and can be bounded analytically for certain restricted classes.

Graph Neural Network embeddings. Training a GNN to embed graphs into a Euclidean space such that $G_i \subseteq G_j$ iff $\mathbf{e}(G_i) \preceq \mathbf{e}(G_j)$ (in some partial-order sense) is an active research area. Order-embedding models [7] have shown promise for relation learning; extending them to subgraph ordering would yield a learned oracle with $\mathcal{O}(n)$ inference time. Key challenges include training data generation, generalisation across graph families, and providing probabilistic guarantees on the error rate.

Randomised algorithms with one-sided error. A randomised algorithm that correctly identifies all true subgraph containment pairs (no false negatives) and produces false positives with probability at most δ would be highly valuable: the HS-DFS meta-graph may have spurious edges, but the DFS forest remains a valid over-approximation of the true containment hierarchy. Designing such algorithms via, e.g., random graph fingerprinting or colour-coding [2] is a natural extension.

Adaptive oracle scheduling. Rather than committing to a single oracle type, an adaptive scheduler could start with a cheap approximate oracle and fall back to the exact Subgraph Algorithm only for pairs flagged as uncertain. Bayesian active learning methods could guide which pairs to re-evaluate at higher cost, minimising the total computational budget for a given accuracy target.

Dynamic and Streaming Graph Collections

The current formulation of HS-DFS assumes a static collection $\mathcal{G} = \{G_1, \ldots, G_N\}$ given in advance. Real-world deployments typically involve collections that evolve continuously.

Incremental meta-graph maintenance. When a new graph G_{N+1} is inserted, only the $2N$ oracle entries involving G_{N+1} need to be recomputed (i.e., whether $G_{N+1} \subseteq G_j$ or $G_j \subseteq G_{N+1}$ for each existing j). The meta-graph can then be updated by adding a new node together with its incident edges. However, the DFS forest may change globally even with a single edge insertion, because new containment relationships can introduce alternative DFS traversal orders. Algorithms for *incremental DFS* on directed graphs, recently developed in the context of online graph algorithms, provide a starting point; adapting them to the HS-DFS setting requires careful treatment of back-edges and cross-edges in the meta-graph.

Graph deletions and expiry. When a graph G_i is removed from the collection, its node and all incident edges must be deleted from $\mathcal{M}$. If G_i was an intermediate node in a containment chain $G_a \subseteq G_i \subseteq G_b$, the transitive closure of the meta-graph must be updated to reflect the direct relationship $G_a \subseteq G_b$. Maintaining the DFS forest under deletions without full recomputation is a challenging open problem.

Streaming with bounded memory. In resource-constrained settings (e.g., edge computing at IoT devices), storing the full oracle matrix is infeasible. A streaming variant of HS-DFS that processes graphs in a single pass and maintains a compact summary – such as a spanning subgraph of $\mathcal{M}$ or a compressed representation of the DFS forest – would enable hierarchical analysis under strict memory constraints.

Temporal containment. Many real networks carry timestamps on edges or evolve according to known temporal patterns (e.g., daily rhythms in brain activity, seasonal supply-chain patterns). Defining *temporal subgraph* containment – where the embedding must respect temporal ordering – and building a corresponding HS-DFS variant for temporal graph streams is an open problem of both theoretical and practical significance.

Weighted, Attributed, and Multi-Relational Graphs

The Subgraph Algorithm and HS-DFS as presented operate on unweighted, unlabelled, directed graphs. Practical graph data is almost universally richer.

Edge and vertex weights. Incorporating weights requires a generalised notion of subgraph containment: an embedding $\varphi : V(G_i) \to V(G_j)$ must preserve not only the edge structure but also weight constraints (e.g., $w_i(u, v) \leq w_j(\varphi(u), \varphi(v))$). The column-signature framework can be extended to encode weight ranges into the signature entries; LCS comparisons must then be replaced by order-respecting matching.

Vertex and edge labels. Labelled graphs arise naturally in cheminformatics (atom types and bond types), semantic web (RDF predicates), and genomics (gene ontology annotations). Label compatibility constraints add a multiplicative factor to the oracle complexity in the worst case, but in practice restrict the search space substantially. The column-signature approach can be adapted by partitioning vertices by label before constructing signatures, maintaining the core algorithmic structure.

Heterogeneous and multi-relational graphs. Knowledge graphs such as Wikidata or Freebase contain multiple distinct edge types ("is-a", "part-of", "located-in", etc.). Subgraph containment in such graphs – where edge types must be matched – is significantly harder; the problem is related to conjunctive query containment, which is Π_2^P-complete in general. Practical algorithms exploiting the sparsity and regularity of real KG schemas are an important direction.

Probabilistic and uncertain graphs. Graphs derived from statistical inference (e.g., brain connectivity from fMRI correlations, gene regulatory network inference) carry inherent uncertainty. Each edge exists with some probability $p_{ij} \in [0, 1]$. Defining *probabilistic subgraph containment* – $G_i \subseteq G_j$ with probability at least $1 - \epsilon$ – and designing HS-DFS variants that propagate uncertainty through the meta-graph constitute a rich open direction.

Hypergraphs, Simplicial Complexes, and Higher-Order Structures

Biological and social networks frequently exhibit *higher-order interactions*: reactions involving three or more molecules simultaneously, group communications, or higher-dimensional topological features. These structures are naturally represented as hypergraphs or simplicial complexes, neither of which is captured by the ordinary graph model.

Hypergraph subgraph containment. A hyperedge connects an arbitrary subset of vertices; a hypergraph H_i is a sub-hypergraph of H_j if there exists an injective map on vertices that sends each hyperedge of H_i to a subset of a hyperedge of H_j. The column-signature approach does not directly generalise, since adjacency matrices are replaced by incidence matrices with variable row sizes. A tensorial encoding of hyperedge memberships may provide a path forward.

Simplicial complex homology. Persistent homology – tracking the birth and death of topological features (connected components, loops, voids) across a filtration – provides a summary of the hierarchical structure of a simplicial complex. Relating the HS-DFS forest to the persistent homology barcode of the collection is a deep and speculative but potentially very fruitful direction: the DFS forest encodes combinatorial containment, while barcodes encode topological containment. Understanding when and how these two notions align would illuminate the structural basis of many hierarchical phenomena in nature.

Integration with Graph Neural Networks and Deep Learning

Graph Neural Networks (GNNs) have revolutionised machine learning on graph-structured data. Their integration with HS-DFS opens several mutually beneficial directions.

HS-DFS as a supervision signal. The containment hierarchy produced by HS-DFS can serve as structured supervision for training GNNs to learn *hierarchical graph representations*: representations in which $\mathbf{e}(G_i)$ is "below" $\mathbf{e}(G_j)$ in embedding space whenever $G_i \subseteq G_j$. Such representations would enable fast approximate lookups in large graph databases without recomputing the oracle. The DFS tree structure provides a natural curriculum: simpler containment relationships (few vertices) can be used for initial training, with progressively deeper relationships added as training proceeds.

GNN expressiveness and the WL hierarchy. The expressive power of standard GNNs is bounded by the 1-dimensional Weisfeiler-Leman test [7]. Since the Subgraph Algorithm uses column signatures derived from the adjacency matrix – which go beyond 1-WL in their sensitivity to global structure via cyclic rotations – it offers a concrete candidate for a "Subgraph-WL" test that may lie strictly between 1-WL and 2-WL in discriminative power. Formalising this relationship and designing GNN architectures that match it would contribute to the ongoing programme of characterising GNN expressiveness.

Meta-graph embeddings for graph retrieval. Given a query graph Q and a large database $\mathcal{G}$, finding all $G_i \in \mathcal{G}$ with $Q \subseteq G_i$ is the *supergraph retrieval* problem, central to drug discovery and code search. The meta-graph $\mathcal{M}$ provides an index that dramatically prunes the search: once Q is located (or approximately located) in $\mathcal{M}$, only ancestors in the DFS forest need to be checked. Training a GNN to predict query position in $\mathcal{M}$ without constructing $\mathcal{M}$ explicitly is a promising direction for scalable graph retrieval.

Transformer architectures on meta-graphs. Attention-based graph transformers operating on $\mathcal{M}$ as the input graph – with nodes as graphs and edges as containment relations – can

learn complex meta-level patterns: e.g., which structural motifs are universally present across a graph collection, or which subgraphs tend to co-occur. This "graph of graphs" view enables a new class of second-order graph learning problems that HS-DFS is uniquely positioned to support.

Quantum Algorithms for Subgraph Problems

Quantum computing offers the prospect of qualitatively different complexity trade-offs for hard combinatorial problems.

Grover-based oracle speedup. Grover's algorithm provides a quadratic speedup for unstructured search. Applied to the permutation search underlying naive subgraph isomorphism, it reduces the search space from $\mathcal{O}(n!)$ to $\mathcal{O}(\sqrt{n!})$ – still super-exponential, but a meaningful improvement. Whether the column-signature structure of the Subgraph Algorithm admits a quantum analogue that achieves better than $\mathcal{O}(n^{1.5})$ oracle complexity is an intriguing open question.

Quantum graph isomorphism. Recent work on quantum query complexity has established non-trivial bounds for graph isomorphism; extending these results to the subgraph setting is largely unexplored. The connection between the cyclic rotation invariance exploited in the Subgraph Algorithm and quantum Fourier transforms over cyclic groups is a structural hint that quantum methods might be particularly natural here.

Variational quantum eigensolver approaches. For small-to-medium graphs, variational quantum algorithms (VQE, QAOA) can be used to encode the subgraph isomorphism problem as a quadratic unconstrained binary optimisation (QUBO) instance and approximately solve it on near-term quantum hardware. Although current hardware is too noisy for a practical advantage, this direction will become increasingly relevant as quantum processors scale.

Parameterized Complexity and Structural Graph Theory

The classical intractability of subgraph isomorphism is alleviated by *parameterized complexity theory*, which identifies structural parameters of the input that, when bounded, admit efficient algorithms.

Tree-width and other width parameters. For pattern graphs of bounded tree-width tw, subgraph isomorphism can be solved in $f(tw) \cdot n^{\mathcal{O}(1)}$ time via Courcelle's theorem (a result in meta-algorithmic form) or more concretely via tree-decomposition dynamic programming. It is natural to ask whether the Subgraph Algorithm can be adapted to exploit bounded tree-width of either the pattern or the host graph, reducing oracle complexity below $\mathcal{O}(n^3)$ in this important special case.

Graph minors and excluded subgraph families. Robertson-Seymour theory guarantees that any minor-closed graph family is characterised by a finite set of excluded minors. For graph collections arising from specific applications (e.g., planar graphs in geographic networks, outerplanar graphs in certain chemical bond structures), the excluded-minor characterisation may enable polynomial-time exact subgraph testing far faster than the general $\mathcal{O}(n^3)$ oracle.

Colour coding and randomized FPT algorithms. For pattern graphs of bounded size k, colour-coding [2] achieves $\mathcal{O}(2^k \cdot n^{\mathcal{O}(1)})$ running time for subgraph detection. When the

collection $\mathcal{G}$ consists of small graphs (e.g., $k = \mathcal{O}(\log n)$), this significantly outperforms the cubic oracle. Integrating color-coding into HS-DFS for the regime of small graphs is a concrete improvement opportunity.

Multi-Layer Networks and Interdependent Systems

Modern infrastructure, social systems, and biological organisms are not captured by a single network: they consist of *multiple layers* of interactions that are coupled to one another.

Multi-layer subgraph containment. A multi-layer graph $\mathbf{G} = (G^{(1)}, \ldots, G^{(L)})$ is a tuple of graphs, one per layer, sharing the same vertex set. A natural generalisation of subgraph containment requires an embedding that is valid simultaneously in all layers. HS-DFS on collections of multi-layer graphs would produce a meta-graph whose edges certify *joint* structural containment – a far stronger relation than containment in any single layer.

Coupled DFS forest. In the DFS phase, the DFS forest of the multi-layer meta-graph may couple containment hierarchies that are independent within each layer. Understanding the resulting structure – when does a node occupy the same position across all layer hierarchies, and when does layer coupling create new hierarchy patterns? – is both theoretically interesting and practically relevant for, e.g., analysing multiplex social networks or multi-omics biological data.

Application to critical infrastructure. Power grids, communication networks, and transportation systems are tightly interdependent. Their joint failure modes are governed by *cascading failures* across layers. HS-DFS applied to a collection of multi-layer subgraphs representing possible failure scenarios could identify which scenarios are structurally contained in others, enabling a partial order over risk scenarios and guiding prioritised mitigation strategies.

Meta-Graph Visualisation and Interactive Exploration

?? introduced initial techniques for rendering the meta-graph. As N and the complexity of $\mathcal{M}$ grow, scalable and meaningful visualisation becomes a significant challenge in its own right.

Hierarchical layout algorithms. The DFS forest is a natural scaffold for a hierarchical graph layout: DFS roots form the topmost level, DFS children descend, and cross-edges and back-edges are rendered as additional arcs. Adapting the Sugiyama layered graph drawing framework to exploit the DFS forest structure – in particular, to minimise edge crossings while preserving the forest's tree sekeleton – is a concrete visualisation research goal.

Interactive drill-down and filtering. A visual analytics tool built on top of HS-DFS should allow a user to click on any node in the meta-graph to inspect the corresponding graph, to filter the view to a subgraph of $\mathcal{M}$ induced by a vertex subset of interest, and to highlight the shortest containment chain between two selected graphs. Integration with existing graph visualisation libraries (Gephi, Cytoscape, D3.js) would facilitate rapid prototyping.

Summary statistics and meta-graph dashboards. For very large collections ($N > 10^4$), interactive node-link rendering is impractical. Statistical summaries – degree distribution of $\mathcal{M}$, DFS forest depth distribution, fraction of DAG vs. tree edges – can be displayed

as dashboards that convey the global shape of the containment hierarchy without rendering every node. Anomaly detection on these statistics could flag unexpected structural changes in a dynamically evolving collection.

Augmented and virtual reality. For collections arising from high-dimensional scientific data (e.g., full connectomes with thousands of brain regions, or molecular databases with tens of thousands of compounds), immersive 3D exploration in augmented or virtual reality environments may offer cognitive advantages over flat 2D representations. Mapping the DFS forest depth to a physical z-coordinate and using spatial audio to encode edge density could exploit human depth perception for rapid hierarchical exploration.

Benchmarking, Open Datasets, and Reproducibility

The empirical evaluation of HS-DFS and competing approaches requires standardised benchmarks that do not yet exist in a unified form.

Synthetic benchmark suites. A systematic benchmark suite should cover the full range of structural parameters: varying N (collection size), n (individual graph size), graph density (sparse to dense), degree of nesting (fraction of true containment pairs), and graph regularity (ranging from random Erdős-Rényi graphs to structured lattices and trees). All benchmarks should be generated by reproducible random models with fixed seeds and published under open licences.

Real-world dataset curation. Each of the application domains discussed in Section 7 provides potential real-world datasets: functional connectome archives (HCP, UK Biobank), metabolic network databases (KEGG [9], BiGG), software dependency graphs (Maven Central, npm), molecular databases (ChEMBL, PubChem), and knowledge graph snapshots (Wikidata dumps). Curating graph collections from these sources, annotating ground-truth containment relationships where available, and publishing them as a community benchmark would greatly accelerate research on hierarchical graph algorithms.

Reproducibility and open-source tooling. The Python implementation of HS-DFS accompanying this paper [1] provides a reference baseline. Future work should encompass: a comprehensive test suite with property-based testing; continuous integration pipelines that enforce correctness on every commit; containerised environments (Docker, Singularity) for noise-free timing experiments; and publication of all experimental artefacts (code, data, logs) under open licences. Adherence to the FAIR principles (Findable, Accessible, Interoperable, Reusable) for research software is essential for meaningful comparative evaluation.

Domain-Specific Optimisations and Deployment

Beyond algorithmic generalisation, each of the application domains from Section 7 offers specific opportunities to tailor and deploy HS-DFS in practice.

Neuroscience and connectomics. The brain's connectome is sparse, hierarchically organised, and exhibits strong bilateral symmetry. Exploiting these priors – for instance, restricting the oracle to connectivity-preserving embeddings or enforcing hemisphere symmetry constraints – can reduce oracle complexity by large constant factors. A dedicated HS-DFS module integrated into existing connectome analysis platforms (e.g., the Human Connectome Project pipeline or Nipype) would enable large-scale comparative connectomics.

Drug discovery and cheminformatics. Molecular graphs contain at most a few hundred atoms; n is small but N (number of compounds in a screen) can reach 10^9 in virtual screening campaigns. An HS-DFS-based scaffold-tree decomposition, computing which molecular fragments are subgraphs of which larger molecules, would provide a principled alternative to existing heuristic scaffold-tree tools (e.g., Scaffold Hunter, RDKit Murcko decomposition) with formal correctness guarantees.

Software engineering and program analysis. Control-flow and call-graph containment between software modules provides a principled basis for *library recommendation*: if a project's dependency graph is a subgraph of that of a well-tested reference project, specific library versions are formally compatible. Integrating HS-DFS into package managers (Cargo, pip, npm) as an optional compatibility checker – invoked at install time to verify structural compatibility – is a concrete deployment scenario.

Knowledge graph completion and alignment. In the semantic web, two ontologies O_1 and O_2 are *aligned* if there exists a containment embedding between their concept graphs. HS-DFS provides not only a binary alignment verdict but the full DFS forest, which encodes the alignment structure at all levels of granularity. Integrating HS-DFS with existing ontology alignment tools (LogMap, AgreementMaker Light) as a post-processing step to verify and structure proposed alignments is a concrete short-term engineering goal.

Theoretical Foundations: Algebra, Order Theory, and Logic

Beyond algorithmic questions, HS-DFS invites a range of deeper theoretical investigations.

The poset of graphs under subgraph containment. The subgraph containment relation (modulo isomorphism) defines a partial order on the set of all finite graphs. The meta-graph $\mathcal{M}$ is a finite induced sub-poset of this infinite structure. Understanding the order-theoretic properties of this sub-poset – its width, its height, the existence of antichains of prescribed size – is a problem in combinatorics with direct implications for the complexity of HS-DFS (e.g., antichains imply no containment edges, making Phase 1 wasteful). Ramsey-theoretic arguments can be used to establish bounds on antichain size for graph collections with prescribed density.

Modal and description logics. Subgraph containment can be expressed in monadic second-order logic (MSO): $G_i \subseteq G_j$ iff there exists an injective function on vertices satisfying the edge-preservation axiom. By Courcelle's theorem, MSO-definable properties are decidable in linear time on graphs of bounded tree-width. Exploring which higher-level properties of the meta-graph (e.g., the number of strongly connected components in $\mathcal{M}$) are MSO-definable and exploiting this for declarative query processing is a direction at the intersection of logic and algorithms.

Category-theoretic formulation. Graph homomorphisms and subgraph embeddings are natural morphisms in the category of graphs. The meta-graph $\mathcal{M}$ is then a representation of part of this category's morphism structure. A categorical framework for HS-DFS – expressing Phase 1 as a functor and the DFS forest as a skeleton of the resulting category – could yield deep structural insights and connections to topos theory, sheaves over graph categories, and compositional approaches to graph rewriting.

Summary of the Research Horizon

Taken together, the directions outlined above span the full spectrum from immediate engineering improvements – parallel implementations, benchmark datasets, domain-specific integrations – to deep theoretical questions at the frontier of algorithms, complexity theory, quantum computing, and mathematical foundations. The unifying theme is that the hierarchical containment structure captured by HS-DFS is not merely an algorithmic artefact but a fundamental aspect of how complex systems are organised: smaller functional units are embedded in larger ones, and understanding this embeddability at scale is essential for science and engineering alike. We therefore anticipate that HS-DFS, and the broader research programme it represents, will stimulate sustained work across multiple communities over the coming years.

References

[1] S. Epp. Der Subgraph Algorithmus. Available at https://gitlab.com/epp-group/subgraph, 2026.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press, Cambridge, MA, 2009.

[3] S. A. Cook. The complexity of theorem-proving procedures. In *Proc. 3rd ACM STOC*, pp. 151–158, 1971.

[4] V. V. Williams. On some fine-grained questions in algorithms and complexity. In *Proc. ICM*, vol. 3, pp. 3431–3472, 2018.

[5] J. R. Ullmann. An algorithm for subgraph isomorphism. *J. ACM*, 23(1):31–42, 1976.

[6] L. P. Cordella, P. Foggia, C. Sansone, and M. Vento. A (sub)graph isomorphism algorithm for matching large graphs. *IEEE TPAMI*, 26(10):1367–1372, 2004.

[7] B. Weisfeiler and A. Leman. The reduction of a graph to canonical form and the algebra which appears therein. *NTI, Series 2*, 9:12–16, 1968.

[8] O. Sporns. *Networks of the Brain*. MIT Press, 2011.

[9] M. Kanehisa and S. Goto. KEGG: Kyoto Encyclopedia of Genes and Genomes. *Nucleic Acids Research*, 28(1):27–30, 2000.

[10] AUTOSAR Consortium. *AUTOSAR Layered Software Architecture*, Release 4.4, 2019.

[11] G. Tononi. Consciousness as integrated information: a provisional manifesto. *Biol. Bull.*, 215(3):216–242, 2008.

[12] S. Epp. Die epistemische Wolke, der unsichtbare Geist des Lebens und die Rechtsdrehung kortikaler Informationsverarbeitung. Available at https://gitlab.com/epp-group/brn, 2026.

[13] A. L. Hodgkin and A. F. Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *Journal of Physiology*, 117(4):500–544, 1952.

[14] J. Del Castillo and B. Katz. Quantal components of the end-plate potential. *Journal of Physiology*, 124(3):560–573, 1954.

[15] P. S. Goldman-Rakic. Cellular basis of working memory. *Neuron*, 14(3):477–485, 1995.

[16] J. J. Radley, H. M. Sisti, J. Hao, A. B. Rocher, T. McCall, P. R. Hof, B. S. McEwen, and J. H. Morrison. Chronic behavioral stress induces apical dendritic reorganization in pyramidal neurons of the medial prefrontal cortex. *Cerebral Cortex*, 14(7):711–717, 2004.

[17] J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982.